Briefing Notes

This activity will take place in DB G.30 and will take 3 hours. You should attend the session shown on your personal timetable. You will be working individually or in groups of 2 and the maximum total number of students is 100.

Health and Safety

The approved Risk Assessment for this activity, reference number: RA_5815, is available on the University LabCup system: https://mmu.uk.labcup.net/.



The class will start with a Health and Safety briefing. Students who miss this will not be allowed to take part until they are briefed - please make sure you arrive in good time.

Attendance Record

You may be required to sign an attendance sheet in the laboratory. Please note that this is for the unit teaching team's local records only and does not replace the requirement for attendance registration for participating students.

Aim

The aim of this experiment is to:

- Introduce the use of modules and packages for additional functionality.
- Introduce the use of the packages: math, cmath, numpy and matplotlib.

Background Reading

- Thonny, Python IDE for beginners: https://thonny.org/
- LearnPython.org interactive Python tutorial: https://www.learnpython.org/

About this resource

Document last updated:	October 2026	*Note that printed or emailed copies of this document may be out of date. Always check Moodle for the latest version.*
Next scheduled review date:	August 2027	
Comments and feedback on this document should be sent to the author(s):		Tim Collins

1 Importing Modules and Packages

1.1 Introduction

To extend the core Python functionality, there are many optional functions and data types that can be used by *importing modules*. A module is a Python script that could be built-in, sourced from a third party or written by yourself. Importing a module runs the script and, so, makes available any functions, data types, variables, etc. that are declared in the script. We used two built-in file formatting modules in the last session: 'csv' and 'json'.

A simple example of a built-in module is the *random* module. This adds the ability for Python to generate pseudorandom numbers and make random choices. For example:

Source Code

```
1 # Run this script several times to see the effects of the random functions.
2 import random
3
4 i = random.randint(3, 8)                    # i = random integer from 3 to 8
5 print(i)
6 c = random.choice(["Red","Green","Blue"])   # selects a random list element
7 print(c)
```

Note that all functions etc. imported from a module must be accessed using a prefix of '*module-name*.' (known as the *namespace* of the module) in this case: `random.`

A module is a single Python file, usually containing a relatively small number of functions. Packages are collections of several module files and can either be imported all together, or individual modules within the package can be selected if the programmer knows they will only need some of the package's full functionality.

2 Mathematics

2.1 The math module

Core Python functionality allows you to do simple arithmetic operations like adding, subtracting, etc. but does not include more sophisticated functions like square roots, trigonometrical functions, exponentials and logarithms, etc. If you need these, you will need to import a mathematics module, the simplest being `math`. For example, to find the square root of a number:

```
1 import math
2
3 x = 25
4 print(math.sqrt(x))
```

Running this script should give the output: `5.0`

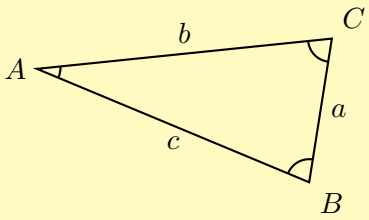
Note that:

- The line: `import math` needs to be executed once before any of the math functions become available.

- All functions used from the math module are in the math namespace and must be prefixed with `math.` so that the interpreter knows where to find them.

A complete list of the functions included in the `math` module can be found here:
https://docs.python.org/3/library/math.html

Exercise: Given the triangle illustrated below:



If: $b = 13\text{cm}, c = 13\text{cm}$ and $A = 0.5$ rad, write a script to calculate the length of the remaining side, a using the cosine rule:

$$a^2 = b^2 + c^2 - 2bc\cos(A)$$

*Hint: Refer to the **Trigonometric functions** section, here:*
https://docs.python.org/3/library/math.html#trigonometric-functions

Exercise: The collector current of a bipolar junction transistor operating in the active region can be approximated by:

$$I_C = I_S \exp\left[\frac{V_{BE}}{V_T}\right]$$

Given a transistor where $I_S = 5 \times 10^{-13}$A and assuming $V_T \approx 26$mV, write a script to calculate the collector current when the base-emitter voltage, $V_{BE} = 0.6$V

*Hint: Refer to the **Power and logarithmic functions** section:*
https://docs.python.org/3/library/math.html#power-and-logarithmic-functions
Also, note that quantities like 5×10^{-13} can be expressed in Python in the form: `5e-13`

Exercise: A signal generator produces a 125Hz sine wave of magnitude 5V defined by:

$$V = 5\sin(2\pi \times 125t)$$

Write a script to print the instantaneous voltage, V, at times t = 0, 1, 2, 3,...,7 ,8 ms
(note that you will need to convert the values of t from milliseconds to seconds)

*Hint: You will need a **Trigonometric function** again, and also the **Constants** section:*
https://docs.python.org/3/library/math.html#constants

2.2 Complex Numbers - cmath

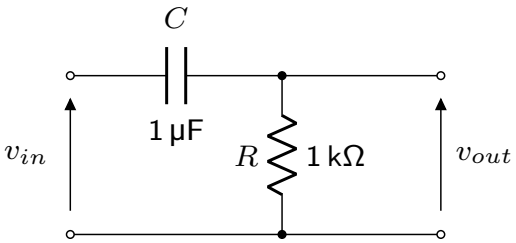
The *math* module provides all the functionality of a basic scientific calculator which is sufficient for many applications. It is not intended to cope with more sophisticated calculations, e.g. involving complex numbers, vectors or matrices.

To use complex numbers, there is an alternative module: *cmath*. It provides the same functions as the math module but with support for complex values. To observe the difference, try the following script:

```
1 import math, cmath                # Import both math and cmath modules
2
3 print(cmath.sqrt(-1))             # Should correctly show the value: 1j
4 print(math.sqrt(-1))              # Will not execute but raises an exception instead
```

In many calculations, a negative square root would be indicative of an error and so using the math module helps. In cases where we expect to have complex answers, though, cmath would, obviously, be required.

As well as providing complex versions of standard mathematical and trigonometric functions, cmath also provides specific functions for handling complex numbers: `cmath.polar()` converts a complex number to its polar form, `cmath.rect()` converts from polar to complex and `cmath.phase()` returns the phase of a complex number in radians.



For example, the circuit above is a first order high-pass RC filter. With a transfer function:

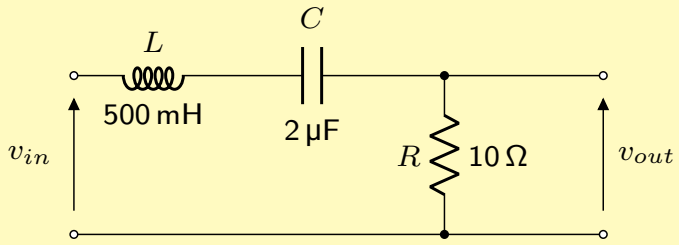
$$H(\omega) = \frac{v_{out}}{v_{in}} = \frac{j\omega CR}{1 + j\omega CR} = \frac{10^{-3}\omega j}{1 + 10^{-3}\omega j} = \frac{j\omega}{1000 + j\omega}$$

In theory, when ω equals the cut-off frequency, $1/CR = 1000\,\text{rad/s}$, the magnitude of the transfer function should be $1/\sqrt{2} \approx 0.707$ and the phase of the transfer function should be 45 degrees. We can check this with the following script:

```
1 import cmath
2
3 def H(w):
4    return 1j*w/(1000 + 1j*w)
5
6 print(f"Magnitude = {abs(H(1000)):.3f}")
7 print(f"Phase = {cmath.phase(H(1000))*180/cmath.pi:.3f} degrees")
```

Note that `abs()` is a built-in Python function but `phase()` and `pi` are contained in the cmath module.

Exercise: The diagram below shows an LCR series resonant circuit



The transfer function of the circuit can be written as:

$$H(s) = \frac{v_{out}}{v_{in}} = \frac{sCR}{s^2LC + sCR + 1}$$

where $s = j\omega$.

In theory, the circuit should resonate at 1000 rad/s with a bandwidth of 20 rad/s. Verify this by writing a script to evaluate the magnitude and phase (in degrees) of the transfer function at:

1. $\omega = 1000\,\text{rad/s}$ *(answers should be 1 and 0)*
2. $\omega = 990\,\text{rad/s}$ *(answers should be approximately 0.707 and 45)*
3. $\omega = 1010\,\text{rad/s}$ *(answers should be approximately 0.707 and -45)*

NB. Since you will need it several times, it is good practice to create a function for the transfer function rather than typing the same code multiple times.

3 Vector Mathematics: numpy and matplotlib

The functions provided by the `math` and `cmath` modules take just a single number at a time as arguments. Often, we want to work with a sequence of numbers and we want to visualise the results. For example, if you wanted to plot a graph of collector current vs base-emitter voltage for the transistor described earlier, using the `math` module would require you to iterate through a list of all the voltage values you want to calculate. For complex systems, this can get slow and cumbersome and the `numpy` package (short for NUMerical PYthon) offers a useful alternative allowing operations on whole vectors or matrices of numbers at once.

3.1 numpy

3.1.1 Installation and Importing

As with any other module or package, `numpy` requires importing before any of its functions can be used. However, if you try this in Thonny you will probably get an error message. Try the following:

Shell

```
import numpy
```

and you will probably see something like this:

```
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
ModuleNotFoundError: No module named 'numpy'
```

The reason for this is that the basic Python installation included with Thonny does not inclued every single package and module. This is reasonable considering there are hundreds of thousands of Python projects available[1]. In order to install `numpy` and make it available using Thonny, follow these steps:

1. In Thonny, from the 'Tools' menu, select 'Manage packages...'
2. A dialogue box should appear - type 'numpy' in the search bar at the top and click 'Search on PyPI'
3. A list of hits should appear, one of which should be numpy (it should be the top hit in the list)
4. Click on numpy in the list
5. A page of information similar to figure 1 should appear - click 'Install'
6. Downloading and installation will proceed and may take a little time. When complete, you should see that `numpy` has been added to the list of installed packages on the left.
7. Click 'Close'

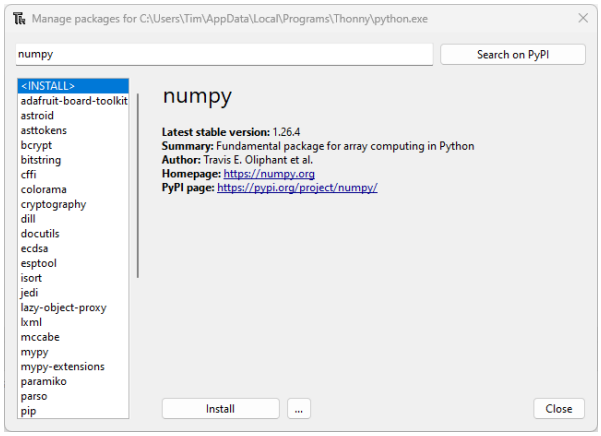


Figure 1: Installing the `numpy` package.

Now, try this again:

```
Shell
import numpy
```

There should now be no errors and numpy is imported successfully.

[1] At the time of writing, PyPI.org lists over 500,000 Python projects.

3.2 Maths with numpy

All the functions found in the math and cmath modules are also included in numpy so it can be used as an alternative mathematics library. However, the point of numpy is its ability to work with arrays of numbers so, if you don't need this functionality, it is generally better to stick with math or cmath which require less processor and memory resources.

3.2.1 Arrays

Imagine you want to calculate the square root of every number between 0 and 9 and store the results in a list. You can do this using the `math` module:

```
1 import math
2
3 y = []
4 for x in range(10):
5     y.append(math.sqrt(x))
6 print(y)
```

Alternatively, list comprehension can be used to create the list, `y`, using a more concise syntax:

```
1 import math
2
3 y = [math.sqrt(x) for x in range(10)]
4 print(y)
```

To do this using the `numpy` package:

```
1 import numpy
2
3 x = numpy.arange(10)
4 y = numpy.sqrt(x)
5 print(y)
```

You should get the same result in each case. The difference is that the `math` version requires a `for` loop to iterate over the values of `x` whereas the `numpy` version processes the whole list in one function call. This results in slightly tidier code and, more importantly, faster execution. You probably won't notice any difference with this simple example but with a bigger range of values to calculate, the execution time difference can be measured and the `numpy` version can be found to execute over 100 times faster.

The `arange()` function defines an array that is an arithmetic progression. More generally, numpy arrays can be defined using the function, `numpy.array()`. This converts a sequence of values into the corresponding array. All elements of the array will be the same data type which will be the simplest one that represents all the input data (usually bool, int, float or complex in that order of preference). For example:

```
1 import numpy
2
3 i = numpy.array([1, 2, 4, 8, 16])      # An array of ints
4 b = numpy.array([True, False, True])   # An array of bools
5 f = numpy.array([3, 2.5, 1, -2])       # An array of floats (due to the 2.5 in the list)
6 c = numpy.array([1+2j, 4, 6])          # An array of complexes (due to element zero)
```

Exercise: Write a script that calculates the logarithm to the base 10 of the sequence:

$$10, 20, 100, 400$$

Use a numpy array and the function `numpy.log10()` to perform the calculation with a single instruction. Print the result - it should be: $1, 1.301, 2, 2.602$

3.2.2 Array Operations and Array Broadcasting

As well as applying functions to all members of an array, you can also use arithmetic operators to perform pairwise calculations on the members of two or more arrays. For example, to add together two lists of numbers:

```
1 import numpy
2
3 a = numpy.array([1, 3, 2, 5])  # a and b are arrays of four ints each
4 b = numpy.array([6, 2, 4, 3])
5 c = a + b                      # Adds each member of a and each member of b
6 print(c)                       # Result: [7 5 6 8]
```

You can also use two of more dimensional arrays is the same way. A 2D array example using subtraction:

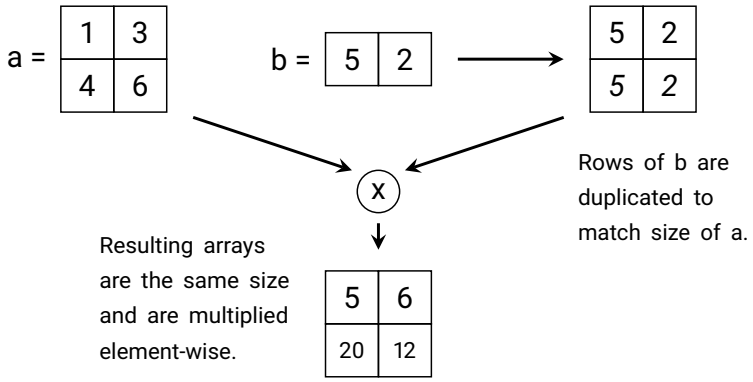
```
1 import numpy
2
3 x = numpy.array([[5, 7], [2, 3]])  # x and y are 2 x 2 arrays
4 y = numpy.array([[2, 3], [5, 1]])
5 z = x - y                      # Subtracts each member of y from each member of x
6 print(z)                       # Result:[[ 3  4]
7                                #         [-3  2]]
```

It is also possible to apply operations with an array and a scalar variable like a float or an int. For example:

```
1 import numpy
2
3 a = numpy.array([4, 6, 5, 2])  # An array of four numbers
4 d = 2
5 e = a / d                      # Divide every element of a by d = 2
6 print(e)                       # Result: [2.0 3.0 2.5 1.0]
```

It can be possible to perform operations with two arrays that are of different shapes or sizes if it is possible for numpy to expand either both, or the smaller of the two arrays, to match the shapes and sizes by a process known as *broadcasting*. For example:

```
1 import numpy
2
3 a = numpy.array([[1,3],[4,6]])  # A 2 x 2 array
4 b = numpy.array([5,2])          # A 1 x 2 array
5 print(a*b)                      # Each row of a is multiplied by b: [[ 5  6]
6                                 #                                    [20 12]]
```



Array broadcasting example

The rules for broadcasting are:

1. If the two arrays differ in their number of dimensions, the shape of the one with fewer dimensions is padded with ones on its leading (left) side (e.g. b went from being a one-dimensional array of size, 2, to a two-dimensional array of size 1 x 2).
2. If the shape of the two arrays does not match in any dimension, the array with shape equal to 1 in that dimension is stretched to match the other shape (e.g. b went from being a 1 x 2 array to a 2 x 2 array).
3. If in any dimension the sizes are unequal but neither is equal to 1, an error is raised.

3.2.3 Matrices and Linear Algebra

A common application for numpy arrays is to perform matrix arithmetic. Matrix addition and subtraction already work using the normal '+' and '-' operators. Matrix multiplication, however, is not the same as the element-wise multiplication performed by the '*' operator. For example, take the matrix multiplication below:

$$\begin{bmatrix} 1 & 3 \\ 2 & 5 \end{bmatrix} \times \begin{bmatrix} 2 & 5 \\ 3 & 6 \end{bmatrix} = \begin{bmatrix} 1 \times 2 + 3 \times 3 & 1 \times 5 + 3 \times 6 \\ 2 \times 2 + 5 \times 3 & 2 \times 5 + 5 \times 6 \end{bmatrix} = \begin{bmatrix} 11 & 23 \\ 19 & 40 \end{bmatrix}$$

Remember, each element in the result of a matrix multiplication is the result of multiplying each element in the corresponding row of the first matrix by the elements in the corresponding column of the second and summing the results. This is not the same as the element-wise multiplication performed by the '*' operator which would give the result:

$$\begin{bmatrix} 1 \times 2 & 3 \times 5 \\ 2 \times 3 & 5 \times 6 \end{bmatrix} = \begin{bmatrix} 2 & 15 \\ 6 & 30 \end{bmatrix}$$

In order to perform matrix multiplication with numpy arrays, a new operator is needed to distinguish it from element-wise multiplication. Matrix multiplication is denoted by the '@' operator.

The example script below illustrates the difference using the matrices from above:

```
1  import numpy
2
3  A = numpy.array([[1, 3],[2, 5]])
4  B = numpy.array([[2, 5],[3, 6]])
5
6  # Element-wise multiplication:
7  print(A*B)                      # [[ 2 15]
8                                  #  [ 6 30]]
9  # Matrix multiplication:
10 print(A@B)                      # [[11 23]
11                                 #  [19 40]]
```

Exercise: Write a script that uses numpy to calculate the matrix, $\mathbf{M}$, from the equation below:

$$\mathbf{M} = \begin{bmatrix} 2 & 5 & 1 & 4 \\ 1 & -2 & 4 & 1 \end{bmatrix} \begin{bmatrix} 3 & 9 & -2 \\ -5 & -2 & -3 \\ 1 & 2 & 4 \\ 6 & -2 & 5 \end{bmatrix}$$

The answer should be: `[[ 6, 2, 5],[23, 19, 25]]`

One of the most significant applications of matrices in engineering problems is in the representation and solution of systems of linear equations. For example, the system of equations:

$$\begin{aligned} x_0 + 3x_1 + 2x_2 &= y_0 \\ 3x_0 - 2x_1 + 3x_2 &= y_1 \\ 4x_0 + x_1 - 2x_2 &= y_2 \end{aligned}$$

Can be written as a single matrix multiplication:

$$\begin{bmatrix} 1 & 3 & 2 \\ 3 & -2 & 3 \\ 4 & 1 & -2 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ y_2 \end{bmatrix}$$

A more compact form would be:

$$\mathbf{A}.\mathbf{x} = \mathbf{y}$$

Where $\mathbf{A}$ is the 3 x 3 matrix of coefficients and $\mathbf{x}$ and $\mathbf{y}$ are the vectors of unknowns x_n and y_n respectively.

To calculate the vector, $\mathbf{y}$, given values of $\mathbf{x}$, the following script could be used. Taking $x_0 = 8, x_1 = 4, x_2 = 2$:

Source Code

```
import numpy

A = numpy.array([[1,3,2],[3,-2,3],[4,1,-2]])   # Coefficient matrix
x = numpy.array([[8],[4],[2]])                 # x0, x1 and x2 as a 3x1 matrix
y = A@x                                        # y = A.x by matrix multiplication
print(y)
```

The result should be a 3 x 1 matrix with values 24, 22, 32.

When solving a system of simultaneous equations like $\mathbf{A}.\mathbf{x} = \mathbf{y}$ from the previous page, it is the vector, $\mathbf{x}$, that is unknown and must be calculated given values for $\mathbf{y}$. This can be calculated by multiplying $\mathbf{y}$ by the inverse matrix of $\mathbf{A}$ but it is usually more efficient to use more direct methods such as Gaussian elimination.

Both approaches are possible using a module within numpy called `linalg` - short for 'linear algebra'. This provides the functions `numpy.linalg.inv()` which can invert a matrix and `numpy.linalg.solve()` to solve a system of equations. It also provides many other functions used in linear algebra (see numpy documentation for more details).

A script that demonstrates both approaches is shown below for values: $y_0 = 11, y_1 = 8, y_2 = 12$.

Source Code

```
import numpy

A = numpy.array([[1,3,2],[3,-2,3],[4,1,-2]])  # Coefficient matrix
y = numpy.array([[11],[8],[12]])              # x0, x1 and x2 as a 3x1 matrix

x_method1 = numpy.linalg.inv(A)@y             # Multiply y by the inverse of A
x_method2 = numpy.linalg.solve(A, y)          # Direct calculation of x given A and y

print(x_method1)                              # Both methods should give the same solution
print(x_method2)                              # x0 = 3, x1 = 2, x2 = 1
```

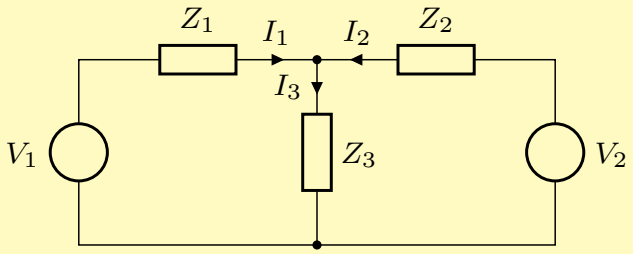
Task: Write a script that uses numpy to calculate the solution of the matrix equation below:

$$\mathbf{A}.\mathbf{x} = \mathbf{y}$$

Where, $\mathbf{A} = \begin{bmatrix} 4 & -2 \\ -1 & 2 \end{bmatrix}$ and $\mathbf{y} = \begin{bmatrix} 2 \\ 7 \end{bmatrix}$

Your script should calculate $\mathbf{x}$ as a numpy array and print the result (as per the CodeGrade template). Enter your code via Moodle to check your solution.

Exercise: The electrical circuit shown below contains two voltage sources and three impedances, the currents are unknown.



Using Kirchhoff's laws, equations relating the currents with the voltage sources and impedances can be formed which can be written in matrix form as:

$$\begin{bmatrix} Z_1 & 0 & Z_3 \\ 0 & Z_2 & Z_3 \\ 1 & 1 & -1 \end{bmatrix} \begin{bmatrix} I_1 \\ I_2 \\ I_3 \end{bmatrix} = \begin{bmatrix} V_1 \\ V_2 \\ 0 \end{bmatrix} \quad \text{or} \quad \mathbf{Z}.\mathbf{I} = \mathbf{V}$$

1. Write a Python function that takes five arguments: `V1`, `V2`, `Z1`, `Z2` and `Z3`, forms numpy arrays to represent the matrices $\mathbf{Z}$ and $\mathbf{V}$, and uses `numpy.linalg.solve()` to calculate the corresponding solution for the matrix, $\mathbf{I}$. Print the solution.
2. Test your function for the following test cases:
   (a) $V_1 = 20\text{V}, V_2 = 26\text{V}, Z_1 = 2\Omega, Z_2 = 4\Omega, Z_3 = 6\Omega$
   (answer should be `[[1.],[2.],[3.]]`)
   (b) $V_1 = 2.6\text{V}, V_2 = 5\text{V}, Z_1 = 100\Omega, Z_2 = 400\Omega, Z_3 = 200\Omega$
   (answer should be `[[0.004],[0.007],[0.011]]`)
   (c) $V_1 = 3\text{V}, V_2 = -6\text{V}, Z_1 = 6\Omega, Z_2 = 22\Omega, Z_3 = 6\Omega$
   (answer should be `[[0.4],[-0.3],[0.1]]`)
   (d) $V_1 = 150\text{V}, V_2 = 150\text{V}, Z_1 = 3\Omega, Z_2 = 2j\Omega, Z_3 = 3\Omega$
   (answer should be `[[16.+12.j],[18.-24.j],[34.-12.j]]`)

3.3 Matplotlib

Matplotlib is a package used for visualising data in publication-quality formats. In engineering mathematics applications, if is useful for viewing calculations performed over a range of values: e.g. time domain plots of waveforms or the frequency response of a circuit. It is also useful in embedded systems applications for visualising sensor data, e.g. plotting temperature variations over time etc.

Like numpy, matplotlib is not included with Thonny by default but can be downloaded in the same way:

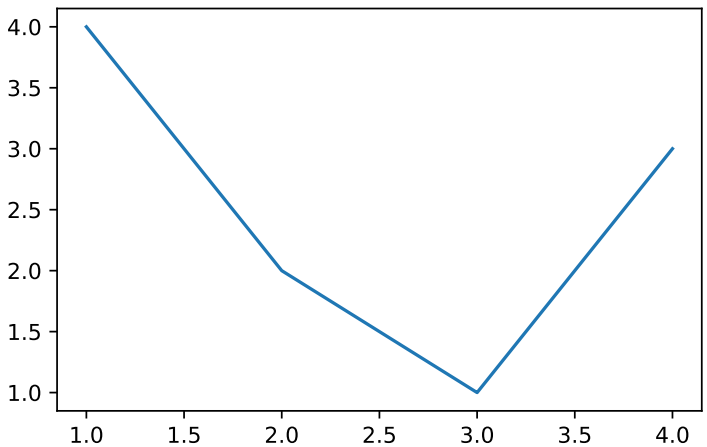
1. In Thonny, from the 'Tools' menu, select 'Manage packages...'
2. In the dialogue box, type 'matplotlib' in the search bar at the top and click 'Search on PyPI'
3. Click on 'matplotlib' in the list
4. A page of information should appear - click 'Install'
5. Downloading and installation will proceed and may take a little time. When complete, you should see that `matplotlib` has been added to the list of installed packages on the left of the dialogue box.
6. Click 'Close'

3.3.1 Plotting Data

Matplotlib is a package containing many modules. We will focus on one for now: `matplotlib.pyplot`.

```
1 import matplotlib.pyplot
2
3 matplotlib.pyplot.plot([1,2,3,4], [4,2,1,3])    # Plot data
4 matplotlib.pyplot.show()                        # Show the plot (don't forget this)
```

This should create a new window and display a plot similar to the one below:



The script uses the function, `matplotlib.pyplot.plot()`, which takes two lists as input arguments. The first list is used for the x coordinates for the plot, the second list for the y coordinates.

In scripts like this, when a module with quite a long name is being used (i.e. `matplotlib.pyplot`) the code can be easier to type and more readable if a shorter alias is used for the module:

```
1 import matplotlib.pyplot as plt     # matplotlib.pyplot is replaced by plt
2
3 plt.plot([1,2,3,4], [4,2,1,3])
4 plt.show()
```

Note the addition of 'as plt' to the import statement. We could have used any valid variable name here but 'plt' is a commonly used choice for the pyplot module. Having created the alias, we can now type just 'plt' instead of 'matplotlib.pyplot' in the rest of the script.

Although two lists were used for the x and y data in this example, the `plot()` function accepts numpy arrays too. For example, to plot a single cycle of a sine wave:

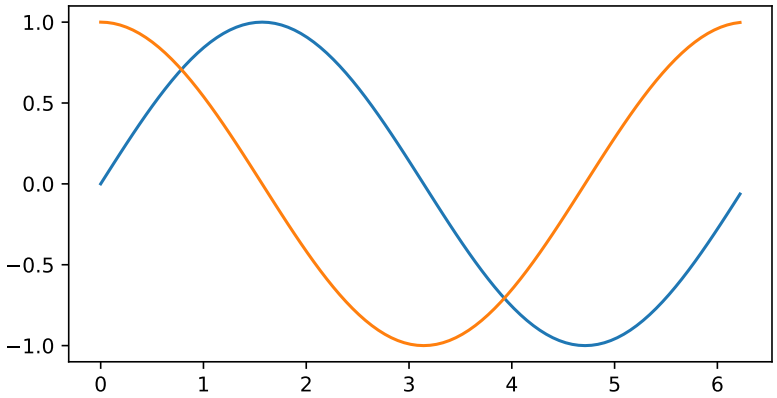
```
1 import numpy
2 import matplotlib.pyplot as plt
3
4 x = numpy.arange(100)              # Creates a numpy array: 0, 1, 2, ..., 99
5 phi = numpy.pi * x / 50            # Scale x to range from 0 to 2.pi
6 plt.plot(phi, numpy.sin(phi))      # x is angle in radians, y is sin(x)
7 plt.show()
```

Like matplotlib.pyplot, it is also common practice to abbreviate numpy as just 'np' which makes the script slightly easier to type. The example below shows this and also shows the effect of adding a second plot.

Source Code

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.arange(100)                # Creates a numpy array: 0, 1, 2, ..., 99
5 phi = np.pi * x / 50              # Scale x to range from 0 to 2.pi
6 plt.plot(phi, np.sin(phi))        # x is angle in radians, y is sin(x)
7 plt.plot(phi, np.cos(phi))        # x is angle in radians, y is cos(x)
8 plt.show()
```

The result should be something like the image below. Note that we now have two plots shown on the same axes as defined in lines 6 and 7 of the script.



3.3.2 Subplots

You may want to plot two or more separate graphs on different axes, but within the same figure. The individual plots are referred to as *subplots*. The procedure is to first create the subplot Axes objects using the `subplots()` function and then plot each graph to the relevant Axes rather than to the figure as a whole. For example, to plot the sine and cosine curves from the last example on separate subplots:

Source Code

```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(100)
phi = np.pi * x / 50
fig, axs = plt.subplots(2,1)    # Create a 2x1 grid of subplot Axes objects
axs[0].plot(phi, np.sin(phi))   # Plot the sine wave to Axes object 0 (top)
axs[1].plot(phi, np.cos(phi))   # Plot the cosine wave to Axes object 1 (bottom)
plt.show()
```

Note that `subplots()` takes two input arguments defining the number of subplots needed in a grid formation; the first argument is the number of rows, the second the number of columns.

Two results are returned from `subplots()`. The first is a `Figure` object that refers to the overall figure and the second is a list of `Axes` objects, each of which can take its own independent plot (or set of plots). If a two-dimensional grid of subplots is requested then a two-dimensional list of `Axes` objects is returned, as in the example below:

Source Code

```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(100)
phi = np.pi * x / 50
fig, axs = plt.subplots(2,2)          # Two rows and two columns specified
axs[0][0].plot(phi, np.sin(phi))      # Top-left (row 0, column 0)
axs[1][0].plot(phi, np.cos(phi))      # Bottom-left (row 1, column 0)
axs[0][1].plot(phi, np.sin(2*phi))    # Top-right (row 0, column 1)
axs[1][1].plot(phi, np.cos(2*phi))    # Bottom-right (row 1, column 1)
plt.show()
```

3.3.3 Labels and Axes Properties

The Axes and Figure objects have several associated methods that can be called to add extra elements to a plot or to alter its appearance. The example below illustrates some of the more commonly used methods to add labels, set the axis limits, add a background grid and set the size of the figure.

Source Code

```
import numpy as np
import matplotlib.pyplot as plt

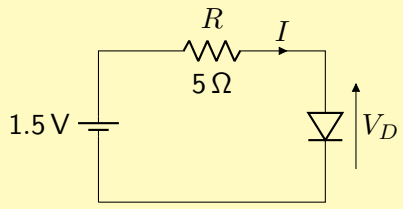
phi = np.linspace(0, 2*np.pi, 50)  # phi array has 50 elements from 0 to 2pi
fig, ax = plt.subplots()           # Only one plot but call this to get fig and ax
```

```
ax.plot(phi, np.sin(phi))          # Plot a single cycle of a sine wave
ax.set_xlabel('Angle')             # Set axis label and plot heading/title
ax.set_ylabel('sin(phi)')
ax.set_title('Sine Wave Example')
ax.set_xlim(0,2*np.pi)             # Set the x and y axis limits exactly
ax.set_ylim(-1,1)
ax.grid(True, color = 'green',     # Enable grid lines and make the dashed
          linestyle = '--')        # and green coloured
fig.set_figwidth(8)                # 8 inches at 100 dpi = 800 pixels wide
fig.set_figheight(4)               # 4 inches at 100 dpi = 400 pixels high
plt.show()
```

Note that this script uses the numpy function, `linspace()`, to produce the `phi` array. This is a quick and easy way to produce a linearly scaled array between two limits.

For a more complete set of examples showing what is possible using matplotlib, take a look through the samples (and their source code) at https://matplotlib.org/stable/gallery/index.html

Exercise: We want to work out the current, I, in the circuit below:



The current, I, can be expressed in two ways as a function of the voltage across the diode, V_D, using the Schockley diode model first, and Ohm's law second:

$$I = I_S \exp\left[\frac{V_D}{nV_T} + 1\right]$$
$$I = (1.5 - V_D)/R$$

Where, I_S = 1e-7 A, $n = 2$, $V_T = 26$mV, $R = 5\Omega$
A graphical way to solve these equations is to plot the two functions and see where they intersect - that must be at the correct value of I and of V_D.

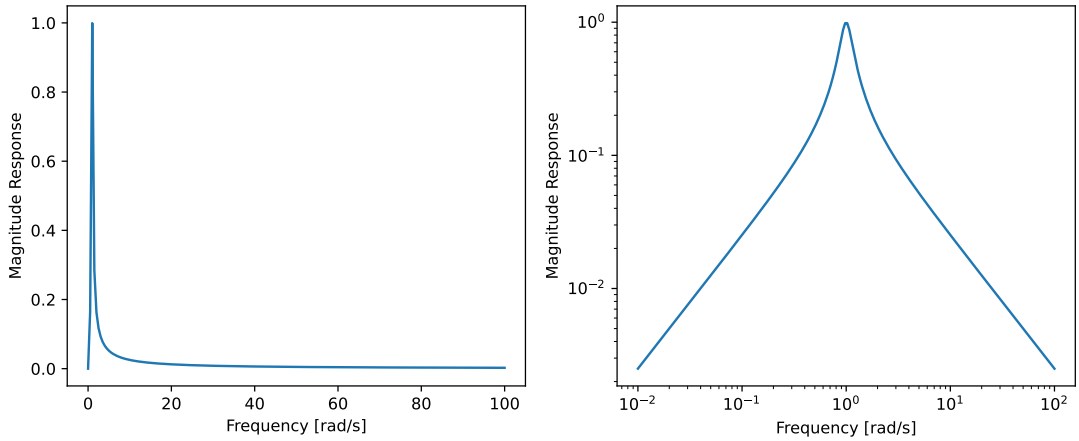
1. Create a numpy array, `VD`, containing 1000 values from 0 to 0.8. You may find the numpy function `linspace()` useful for this.
2. Create two more numpy arrays, `I1` and `I2`, calculated from `VD` using the two equations above.
3. Plot the two arrays, `I1` and `I2`, against `VD` using matplotlib.
4. Use the zoom controls of the plot to estimate the intersection point of the two plots.

3.3.4 Log Axes

In Electronic Engineering, we often use logarithmic, rather than linear, scales for the x and/or the y axes. For example, taking a bandpass filter with the transfer function:

$$H(s) = \frac{s}{4s^2 + s + 4} \qquad \text{(where } s = j\omega\text{)}$$

The figure below shows the magnitude of this transfer function against frequency on linear and logarithmic scales. With the linear scale (left) the detail of the the passband region is compressed into a small width due to the linear frequency axis and the nature of the roll-off away from resonance is hard to see due to the linear magnitude axis. Both issues are remedied by using logarithmically scaled axes (right).



The same transfer function plotted linearly (left) and logarithmically (right)

To produce the logarithmic plot, the script below can be used. Note that the numpy function `logspace()` is used to create the frequency array - recommended when logarithmic plots are to be produced. The limits of `logspace()` are specified as $\log_{10}$ of the actual array limits so, in this case, the limits are 10^{-2} and 10^{2} (i.e. 0.01 and 100).

Source Code

```
import matplotlib.pyplot as plt
import numpy as np

fig,ax = plt.subplots()                 # Called so we can set labels later

w = np.logspace(-2,2,200)               # Log array between 0.01 and 100
s = 1j * w                              # Calculate s = jw
ax.loglog(w, np.abs(s/(4*s**2+s+4)))    # loglog() uses a log x and a log y axis

ax.set_xlabel('Frequency [rad/s]')
ax.set_ylabel('Magnitude Response')
plt.show()
```

Sometimes, only one axis needs to be logarithmic and the other linearly scaled. A common example related to the case above is if you want to plot the response in decibels by calculating $20\log_{10}(|H(s)|)$ first. Now we still want a logarithmic x axis for frequency but the y axis (decibels) should be linear. The function `semilogx()` does this:

Source Code

```
import matplotlib.pyplot as plt
import numpy as np

fig,ax = plt.subplots()                      # Called so we can set labels later

w = np.logspace(-2,2,200)                    # Log array between 0.01 and 100
s = 1j * w                                   # Calculate s = jw
H_dB = 20*np.log10(np.abs(s/(4*s**2+s+4)))   # Calculate H and convert to dB
ax.semilogx(w, H_dB)                         # Log x axis, linear y axis

ax.set_xlabel('Frequency [rad/s]')
ax.set_ylabel('Magnitude Response [dB]')
plt.show()
```

Exercise: A low-pass filter with a cut-off frequency of 1 kHz has the transfer function:

$$H(s) = \frac{4\text{e}7}{s^2 + 8900s + 4\text{e}7}$$

Using matplotlib, create a plot of the magnitude and phase responses of the filter using a logarithmic frequency scale between 10 Hz and 100 kHz (i.e. 10^1 and 10^5). Plot the magnitude response in decibels and the phase response in degrees. To do this:

1. Create a numpy array, `f`, using the `logspace()` function between 10^1 and 10^5.
2. Create another array, `s`, where: $s = j2\pi f$
3. Calculate the transfer function, $H(s)$, using the equation above.
4. Calculate the magnitude in decibels and store in another array: `H_dB` (see example above)
5. Calculate the phase using the functions `numpy.angle()`, to find the phase in radians, and then `numpy.degrees()` to convert the answer to degrees. Store in another array: `H_phase`.
6. Create a grid of two matplotlib subplots (one above the other) and use the `semilogx()` method of each Axes object to plot the magnitude and phase against frequency, `f`.

Features you should be able to note from the plot are:

1. At low frequencies, the magnitude response is flat at 0 dB and the phase is close to 0°.
2. At the cut-off frequency, 1 kHz, the magnitude should fall to -3 dB and the phase to -90°.
3. At high frequencies the magnitude should fall-off at a constant rate of 40 dB per decade of frequency and the phase should approach -180°.